



PRE-LAB 01

SQL — SELECT Statement & WHERE Clause

Course Instructor: Dr. Ahmed Hassan Butt

1. Objectives

This is the first lab of the course, so this pre-lab is designed to build a solid foundation before you start writing SQL on your own. By the end of this pre-lab, you should be able to:

- Retrieve data from a table using the SELECT statement
- Fetch all columns using * or specific columns by name
- Remove duplicate records from a result set using DISTINCT
- Filter rows based on conditions using the WHERE clause
- Apply Comparison, Logical, Arithmetic, and Bitwise operators inside a query
- Sort query results using ORDER BY and restrict the number of rows returned using LIMIT

2. Instructions

- This is a practice pre-lab only. The actual database that will be used in your course lab has not been loaded into the system yet.
- A small dummy database based on a YouTube-style video platform has been created for this document, purely so you can practice writing queries before the real lab.
- All table names (Users, Channels, Videos, Subscriptions, Comments) and column names used in the questions belong to this dummy database. When your real database is loaded in the lab, apply the same query logic using the actual table and column names given to you.
- For every question, write your query and run it — you will compile your queries and screenshots into a separate document as described in the Submission Instructions at the end.
- Once completed, submit your solved document as instructed in Section 9.

3. Getting Started with PostgreSQL

If this is your first time using PostgreSQL, follow these steps carefully, in order. They will take you from opening the software to running your very first query.

1. Open pgAdmin (installed along with PostgreSQL). When it opens, enter the master password you set during installation.
2. In the left sidebar, expand Servers → PostgreSQL. Enter the password for the 'postgres' user when prompted to connect to the server.
3. Right-click on Databases → Create → Database. Name it something like youtube_lab and click Save. This creates an empty database for you to work in.
4. Click on your new database, then open the Query Tool (Tools → Query Tool, or the lightning-bolt icon). This is where you will type and run all your SQL commands.
5. Run the setup script described in Section 6 below. This creates all five tables and loads them with sample data in a single step.

- For each pre-lab question: clear the Query Tool, type your SQL query, and execute it. The results will appear in the Data Output panel at the bottom of the screen.
- Take a screenshot showing both your query and its output. On Windows, press Windows + Shift + S to select and copy an area of the screen; on Mac, press Command + Shift + 4. Then paste (Ctrl+V / Command+V) the screenshot into your separate submission document, under the matching question number, as described in the Submission Instructions.

TIP

If a query returns an error, read the error message carefully — it usually tells you exactly which word or column PostgreSQL did not understand.

4. Scenario

We will use a mini YouTube-style video platform database for this lab. It models Users, Channels, Videos, Subscriptions, and Comments — five tables that together represent how a video platform keeps track of its users, the channels they own, the videos those channels publish, who subscribes to whom, and the comments left on videos.

5. Entity Relationship Overview

- A User can own many Channels (1:M). Every Channel stores its owner in a `user_id` column — the owner of a channel is simply a user, so there is no separate `owner_id` field.
- A Channel can publish many Videos (1:M).
- A User can subscribe to many Channels, and a Channel can have many subscribers → many-to-many, resolved by the Subscriptions table. Each (`user_id`, `channel_id`) pair is unique, so a user cannot subscribe to the same channel more than once.
- A User can write many Comments, and a Video can receive many Comments (1:M, 1:M).

All tables have a PRIMARY KEY, and all relationships are enforced using FOREIGN KEY constraints — satisfying entity integrity and referential integrity, the core properties of a relational database.

6. Database Setup Script (PostgreSQL)

Database Setup — Use the Provided Script

The table creation and sample data have been packaged into a single setup script (**setup-db.sql**) that will be shared with you separately.

- Open the Query Tool for your database (Tools → Query Tool).
- Open the `setup-db.sql` file directly in the Query Tool (File → Open), or drag it in.
- Click Execute/Run (the play icon, or F5) once. This single script drops any old tables, creates all five tables with their keys and constraints, and loads all sample data in one go.
- Do not copy-paste tables or data definitions from this document — running the script is all that's required before you start Section 8.

7. Schema Reference

The setup script creates the following tables. You do not need to type these definitions anywhere — this is only so you know which columns are available while writing your queries.

USERS

Column	Description
<code>user_id</code>	SERIAL, PRIMARY KEY
<code>name</code>	VARCHAR(50), NOT NULL
<code>email</code>	VARCHAR(100), NOT NULL, UNIQUE

Column	Description
country	VARCHAR(50)
joined_date	DATE, NOT NULL

CHANNELS

Column	Description
channel_id	SERIAL, PRIMARY KEY
channel_name	VARCHAR(100), NOT NULL
category	VARCHAR(50)
subscriber_count	INT, DEFAULT 0
user_id	INT, NOT NULL — the owning user (FOREIGN KEY → Users.user_id). The owner of a channel is a user, so this column is named user_id, not owner_id.
status	VARCHAR(20)
created_date	DATE, NOT NULL

VIDEOS

Column	Description
video_id	SERIAL, PRIMARY KEY
title	VARCHAR(150), NOT NULL
category	VARCHAR(50)
views	INT, DEFAULT 0
ad_revenue	NUMERIC(8,2), NOT NULL
duration_seconds	INT
upload_date	DATE
channel_id	INT, NOT NULL (FOREIGN KEY → Channels.channel_id)

SUBSCRIPTIONS (M:N RESOLVER BETWEEN USERS AND CHANNELS)

UNIQUE constraint on (user_id, channel_id) — a user can only subscribe to the same channel once.

Column	Description
subscription_id	SERIAL, PRIMARY KEY
user_id	INT, NOT NULL (FOREIGN KEY → Users.user_id)
channel_id	INT, NOT NULL (FOREIGN KEY → Channels.channel_id)
subscribed_date	DATE, NOT NULL

COMMENTS

Column	Description
comment_id	SERIAL, PRIMARY KEY
video_id	INT, NOT NULL (FOREIGN KEY → Videos.video_id)
user_id	INT, NOT NULL (FOREIGN KEY → Users.user_id)

Column	Description
rating	INT, CHECK (rating BETWEEN 1 AND 5)
likes	INT, DEFAULT 0
comment_text	VARCHAR(255)
comment_date	DATE, NOT NULL

8. Pre-Lab Questions

The table and column names used below (Users, Channels, Videos, Subscriptions, Comments) belong to the dummy YouTube-style database created above, since your real lab database has not been loaded yet. Use these exact names while practicing here — once your actual database is provided in the lab, apply the same query logic using your real table and column names.

- Q1. Write a query to display all columns and all rows from the Videos table.
- Q2. Write a query to display only the title, category, and views columns from the Videos table.
- Q3. Write a query to display the unique categories present in the Videos table (avoid duplicates).
- Q4. Write a query to display all videos that have more than 500000 views.
- Q5. Write a query to display all users who are not from 'Pakistan'.
- Q6. Write a query to display all channels that have a status of 'Active' and more than 100000 subscribers.
- Q7. Write a query to display all videos that belong to either the 'Comedy' or 'Vlog' category.
- Q8. Write a query to display the title and ad_revenue of every video, along with a new calculated column showing the revenue after a 10% platform fee ($\text{ad_revenue} - \text{ad_revenue} * 0.10$).
- Q9. Write a query to display the comment_id and likes of every row in Comments, along with a new column that uses the bitwise AND operator ($\text{likes} \& 1$) to help determine whether the likes value is odd or even.
- Q10. Write a query to display the top 3 videos with the highest views, showing only title and views, sorted from highest to lowest.

9. Submission Instructions

- Solve each of the 10 questions above using the dummy YouTube-style database created via the setup script in Section 6.
- For every question, write the SQL query and run it in the Query Tool, then take a screenshot showing both the query and its output.
- Create a new Word document and paste each question's query and screenshot into it, clearly labeled Q1 through Q10, in order.
- Name the file using your Roll Number followed by PreLab01 (e.g., RollNo_PreLab01.docx).
- The Google Form for submission will be shared later.
- This lab is graded and carries 30 marks.

DEADLINE: 14th September, 1:30 PM

NOTE

This pre-lab uses dummy table names since your real lab database has not been loaded yet. Apply the same query logic using your actual table and column names once the real database is provided in the lab.